Agents_v1

Agentic Data Pipeline for Polymarket Crypto Markets

Project Documentation · March 2026

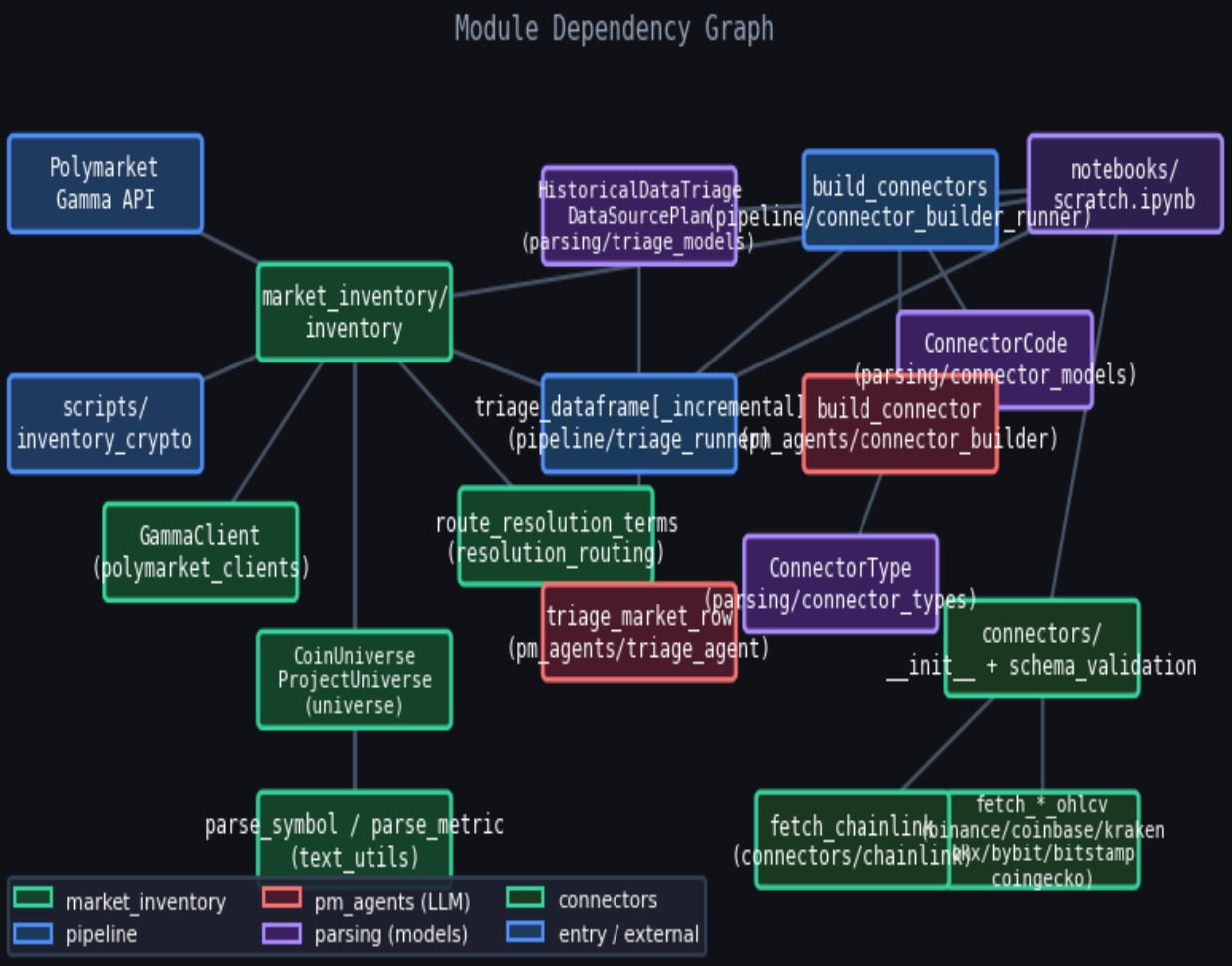
Overview

Agents_v1 is a research scaffold that automates the sourcing of historical price and metric data for active crypto prediction markets on Polymarket. It follows a three-stage pipeline:

- **Stage 1 – Market Inventory:** Queries the Polymarket Gamma API, filters for crypto edge/range markets, and normalises metadata (symbol, metric, resolution source, resolution terms, data type, interval).
- **Stage 2 – LLM Triage:** An OpenAI agent (gpt-5.4) assesses each market's historical-data relevance, feasibility, and paywall risk, and proposes concrete DataSourcePlan objects describing where and how to obtain the data.
- **Stage 3 – Connector Building:** A second LLM agent generates working Python connector functions from each DataSourcePlan. Results are deduplicated by connector_key and persisted in a JSON registry.

A triage cache (parquet) records all triage outputs keyed by market question. Subsequent pipeline runs diff the live inventory against the cache and re-triage only new or changed rows, making repeated runs cheap.

Module Dependency Graph



Arrows indicate import / call dependencies. Colour bands represent logical layers.

Package: market_inventory

Responsible for discovering and normalising active crypto markets from Polymarket. Outputs a DataFrame that feeds the triage pipeline.

Key Classes

Class	Module	Description
`GammaClient`	`polymarket_clients`	HTTP wrapper for the Polymarket Gamma REST API (events, tags).
`ClobClient`	`polymarket_clients`	HTTP wrapper for the Polymarket CLOB API (midpoint prices).
`CoinUniverse`	`universe`	Frozen dataclass: coin symbol + name-to-symbol lookup loaded from coins_universe.json.
`ProjectUniverse`	`universe`	Frozen dataclass: project key/label lookup with fuzzy phrase matching.
`ResolutionRouting`	`resolution_routing`	Frozen dataclass carrying (data_type, interval, interval_source, notes) routing decision.

Key Functions

Function	Module	Description
`inventory_crypto_markets()`	`inventory`	Main entry point. Pages through Gamma API events, filters edge/range markets, extracts symbol, metric, resolution source/terms/date, routing. Returns DataFrame.
`classify_edge_or_range()`	`inventory`	Classifies a market question + outcomes as 'edge', 'range', or 'unknown' using outcome-text heuristics.
`extract_resolution_source_and_terms()`	`inventory`	Extracts resolution URL/provider and full resolution terms from market/event JSON.
`route_resolution_terms()`	`resolution_routing`	Deterministic regex router: maps resolution terms → ResolutionDataType + interval.
`parse_underlying_symbol()`	`text_utils`	Extracts the underlying asset symbol (e.g. BTC, ETH) from the market question.
`parse_metric()`	`text_utils`	Extracts the metric being measured: price, fdv, market_cap, dominance, tvl, etc.

Package: parsing

Pydantic models and enumerations that define the data contracts between the triage agent, the connector builder agent, and the pipeline runners.

Key Classes

Class	Module	Description
`ConnectorType`	`connector_types`	String enum: FREE_API_GENERIC, WAYBACK_SNAPSHOTS, PAYWALLED_PROVIDER, and 12 others.
`DataCandidate`	`historical_data_triage_models`	One candidate historical series: name, unit, frequency, proxy_ok, proxy_notes.
`DataSourcePlan`	`historical_data_triage_models`	Concrete acquisition plan: connector_type, connector_key, series_id, method, target, url_or_endpoint_hint, access, effort, reliability, extraction_target, extraction_method_detail, output_columns, connector_function_name, …
`HistoricalDataTriage`	`historical_data_triage_models`	Full triage output for one market: historical_relevance, data_feasibility, paywall_risk, candidates, plans, recommended_resolution, routing_notes.
`ConnectorCode`	`connector_models`	Generated connector artefact: source_code, imports, dependencies, output_columns, connector_key, series_id, notes.

Package: pm_agents

Two LLM-backed agents built on the OpenAI Agents SDK, both using model **gpt-5.4**. They produce structured Pydantic outputs enforced via strict JSON schema.

historical_data_triage_agent

Given a market row dict (question, symbol, metric, resolution_source, resolution_terms, resolution_data_type, resolution_interval, …), the agent produces a *HistoricalDataTriage* with relevance/feasibility/paywall judgements and a list of DataSourcePlans.

Symbol	Type	Purpose
`historical_data_triage_agent`	`Agent`	Agent instance. Model: gpt-5.4. Output type: HistoricalDataTriage.
`triage_market_row(row, timeout_s)`	`async fn`	Calls the agent for a single row dict. Returns HistoricalDataTriage.

connector_builder_agent

Given a DataSourcePlan, the agent generates a self-contained Python connector function with import statements, source code, and dependency list.

Symbol	Type	Purpose
`connector_builder_agent`	`Agent`	Agent instance. Model: gpt-5.4. Output type: ConnectorCode.
`build_connector(plan, timeout_s)`	`async fn`	Calls the agent for a single DataSourcePlan. Returns ConnectorCode.

Package: pipeline

Orchestration runners that fan out agent calls with concurrency control, timeouts, error handling, and caching.

historical_triage_runner

Function / Variable	Description
`DIFF_COLUMNS`	List of 12 columns used to detect whether an inventory row changed vs cache.
`DEFAULT_CACHE_PATH`	Default parquet path for the triage cache ('triage_cache.parquet').
`save_triage_cache(df, path)`	Persist a triaged DataFrame to parquet.
`load_triage_cache(path)`	Load the triage cache parquet; returns None if absent.
`diff_inventory_vs_cache(inventory_df, cache_df)`	Compare live inventory vs cache. Returns only new or changed rows.
`triage_dataframe_async(df, ...)`	Fan-out async triage: one agent call per row, semaphore-bounded, optional total timeout.
`triage_dataframe_incremental_async(inventory_df, ...)`	Load cache → diff → triage delta → merge → save cache → return merged DataFrame.
`triage_dataframe[_incremental](...)`	Sync wrappers via asyncio.run().

connector_builder_runner

Function	Description
`build_connectors_async(triaged_df, ...)`	Extracts DataSourcePlans from triage_plans_json, deduplicates by connector_key, fans out build_connector calls.
`build_connectors(...)`	Sync wrapper.
`save_registry(connectors, path)`	Persist Dict[connector_key → ConnectorCode] to JSON.
`load_registry(path)`	Load connector registry from JSON.
`write_connectors_module(connectors, path)`	Write all generated connectors to a single .py module with deduplicated imports.

Package: connectors

Hand-built, schema-validated OHLCV connectors for the seven standard Polymarket resolution sources, plus an on-chain Chainlink connector. Each connector returns a DataFrame with columns **timestamp, open, high, low, close, volume**.

OHLCV Connectors

Function	Source	Endpoint
`fetch_coingecko_ohlcv()`	CoinGecko	/coins/{id}/ohlc + /market_chart
`fetch_binance_ohlcv()`	Binance	/api/v3/klines
`fetch_coinbase_ohlcv()`	Coinbase Exchange	/products/{id}/candles

`fetch_kraken_ohlcv()`	Kraken	/0/public/OHLC
`fetch_bitstamp_ohlcv()`	Bitstamp	/api/v2/ohlc
`fetch_okx_ohlcv()`	OKX	/api/v5/market/candles
`fetch_bybit_ohlcv()`	Bybit v5	/v5/market/kline

Schema Validation

Class / Function	Description
`ColumnSpec`	Describes one expected column: name, dtype_kind, nullable, min/max value.
`SchemaSpec`	Full connector schema: expected columns, min_rows, probe_kwargs, OHLC sanity flags.
`ValidationResult`	Outcome: ok flag, errors list, warnings list, actual columns/dtypes, row_count. summary() → str.
`validate_schema(df, spec)`	Validates a DataFrame against a SchemaSpec.
`probe_connector(spec, fn, **kwargs)`	Calls the connector function and validates its output in one step.
`SCHEMA_REGISTRY`	Dict mapping connector name → (SchemaSpec, fetch_fn). Used by check_all_connectors().
`check_all_connectors()`	Probes every registered connector and returns Dict[name, ValidationResult].

End-to-End Data Flow

Step	Function / Class	Output
1	`inventory_crypto_markets()`	DataFrame: market, kind, symbol, metric, resolution_date, resolution_source, resolution_terms, resolution_data_type, resolution_interval, routing_notes, …
2	`diff_inventory_vs_cache()`	Subset of inventory rows that are new or changed vs parquet cache.
3	`triage_dataframe_incremental_async()`	DataFrame with triage columns appended: historical_relevance, data_feasibility, paywall_risk, triage_plans_json, triage_candidates_json, triage_routing_notes, triage_error.
4	`build_connectors_async()`	Dict[connector_key → ConnectorCode] with generated Python source, imports, and dependencies.
5	`write_connectors_module()`	Single .py module containing all generated connector functions, ready to import.

The triage cache (parquet) and connector registry (JSON) provide persistence between runs. On subsequent invocations only the delta is sent to the LLM, keeping API costs proportional to the rate of market change rather than to the total number of active markets.